Bancos de Dados Não-Relacionais

Aula 06: Do CRUD Básico e Python (`PyMongo`) à Modelagem de Documentos BSON

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

27 de Outubro de 2026

INSTITUTO FEDERAL
Ceará
Campus Tauá

Agenda do Encontro

Fundamentos Teóricos: Do RDBMS ao Modelo de Documentos e BSON

Aprendendo do Zero: CRUD Passo a Passo no Compass e `mongosh`

Programando com o Driver Oficial Python (`PyMongo`)

Teoria de Schema Design: *Embedding*, *Referencing* e Padrões

Modelando o E-Commerce ShopGrids no Compass e no Python

Fundamentos Teóricos: Do RDBMS ao Modelo de Documentos e BSON

Do RDBMS ao Modelo Orientado a Documentos

Por que Normalizamos no SQL (1NF a 5NF)?

- Em bancos relacionais, se guardarmos dados do aluno e matrículas na mesma tabela, surgem **redundâncias** e **anomalias de atualização** (*Update Anomalies*): alterar o endereço do aluno exige atualizar várias linhas.
- As **Formas Normais (1NF–5NF)** resolvem isso fragmentando os dados em dezenas de tabelas ligadas por chaves estrangeiras.
- **O Custo:** Para montar uma única tela da aplicação, o banco precisa executar múltiplos `JOINs` pesados em disco!

A Filosofia do Modelo de Documentos

- No MongoDB, a unidade básica não é uma linha plana, mas um **Documento BSON** rico que suporta até **100 níveis de aninhamento** e **16 MB**.
- **Modelagem Guiada pela Aplicação:** Em vez de fragmentar uma entidade em 5 tabelas, agrupamos os dados que pertencem ao mesmo objeto em subdocumentos e arrays.
- **Ganho Direto:** Leitura de objetos complexos em **1 única operação de I/O** e atomicidade nativa no documento!

INSTITUTO FEDERAL
Ceará
Campus Tauá

Terminologia SQL vs. MongoDB e Ecossistema de Ferramentas

Conceito SQL	MongoDB	Definição Técnica
`Database`	`Database`	Agrupador lógico (`mongodb_books`, `ecommerce_shopgrids`).
`Table`	`Collection`	Conjunto de documentos com esquema dinâmico/flexível.
`Row / Tuple`	`Document` (BSON)	Objeto serializado (até **16 MB** e **100 níveis**).
`Column`	`Field` (Campo)	Par `chave: valor` (escalares, objetos e arrays).
`Primary Key`	Campo `_id`	Chave obrigatória e única (`ObjectId` de 12 bytes).
`JOIN`	**Embedding** / `$lookup`	Aninhamento direto ou agregação entre coleções.

Ferramentas Oficiais: 1. MongoDB Compass (GUI visual) • **2. `mongosh`** (Shell JS/Node.js) • **3. Drivers** (`PyMongo`).

JSON vs. BSON (*Binary JSON*): Os 3 Pilares de Projeto

Por que o MongoDB Usa BSON no Motor?

- O JSON puro é texto: exige *parsing* lento caractere por caractere e só possui um tipo numérico (`number` IEEE 754 float, onde $0.1 + 0.2 = 0.30000000000000004$).
- O formato **BSON** foi projetado sobre **3 características fundamentais**:
  1. ***Lightweight* (Leve):** Baixa sobrecarga espacial para tráfego de rede.
  2. ***Traversable* (Percorrível):** Cabeçalhos com tamanho em bytes permitem saltar campos em $\mathcal{O}(1)$!
  3. ***Efficient* (Eficiente):** Codificação e decodificação rápidas em tipos nativos de máquina (C/C++).

Principais Tipos de Dados BSON

- **`ObjectId`:** Identificador binário de 12 bytes rápido de gerar e ordenado no tempo.
- **`String`:** Cadeia de caracteres UTF-8.
- **`Object` e `Array`:** Subdocumentos embutidos e listas de valores.
- **`Int32` e `Int64` (`Long`):** Inteiros exatos de 32 e 64 bits.
- **`Double`:** Ponto flutuante de 64 bits.
- **`Decimal128`:** Decimal de 128 bits com precisão exata para valores monetários e fiscais!
- **`Date` (`ISODate`):** Inteiro de 64 bits (ms desde a Unix Epoch).

Anatomia do `ObjectId` (12 Bytes) e Sintaxe no Compass (EJSON)

1. Estrutura dos 12 Bytes do `ObjectId`



- Dispensa `AUTO_INCREMENT` centralizado: pode ser gerado pelo próprio cliente Python!
- Como os 4 primeiros bytes são o *timestamp* Unix, ordenar por `_id` já ordena por ordem de criação!

2. Como Inserir no Compass (*Extended JSON*)

```
// Na janela Insert Document do Compass, todas
// as chaves exigem aspas duplas ("..."):
{
  "_id": { "$oid": "671ec880a1b2c3d4e5f60001"
         },
  "sku": "NOTE-ACER-15",
  "nome": "Notebook Gamer Nitro V15",
  "preco": { "$numberDecimal": "4599.90" },
  "estoque": 15,
  "ativo": true
}
```

- `"$oid"` → `ObjectId`;
`"$numberDecimal"` → `Decimal128`.

Aprendendo do Zero: CRUD Passo a Passo no Compass e mongosh

Passo 1: Criando o Banco e Consultando Coleções

1. Pela Interface Gráfica (MongoDB Compass)

1. Conecte em `mongodb://localhost:27017`.
2. No menu esquerdo (Databases), clique em **+ (Create database)**.
3. Preencha **Database Name:** `mongodb_books` e **Collection Name:** `books`.
4. **Consulte a lista de coleções:** Clique sobre `mongodb_books` no menu lateral e verifique a coleção `books` na aba **Collections**.

2. Pelo Terminal Interativo (`mongosh`)

```
// Listar todos os bancos existentes:
show dbs

// Selecionar/criar o banco mongodb_books:
use mongodb_books

// Criar explicitamente a colecao books:
db.createCollection("books")

// Consultar a lista de colecoes do banco:
show collections
// Equivalente em JavaScript:
db.getCollectionNames()
```

Passo 2: Inserindo (Create) e Consultando (Read) o 1.º Documento

1. Inserção Manual no MongoDB Compass

Em `books`, clique em **Add Data** → **Insert Document**, cole o JSON e clique em **Insert**:

```
{
  "isbn": "101",
  "name": "MongoDB 7.0 Guia Pratico",
  "price": 30
}
```

- **Consultar a lista de documentos:** Na aba **Documents**, veja que o MongoDB gerou o campo `_id` (`ObjectId`) automaticamente! Alterne entre as visões **List**, **JSON** e **Table**.

2. Equivalente no `mongosh` (`insertOne` e `find`)

```
// Inserir 1 documento:
db.books.insertOne({
  isbn: "101",
  name: "MongoDB 7.0 Guia Pratico",
  price: 30
})
// Retorno:
// { acknowledged: true,
//   insertedId: ObjectId("671ec880...") }

// Consultar todos os documentos da colecao:
db.books.find()
```

Passo 3: Subdocumentos Aninhados e Busca com *Dot Notation*

1. Inserindo Documento com Objeto `meta`

Em **Add Data** → **Insert Document** no Compass (após inserir, consulte a lista de documentos e expanda `meta`):

```
{
  "isbn": "201",
  "name": "Modelagem de Documentos NoSQL",
  "meta": {
    "version": "MongoDB 6.0"
  },
  "price": 50
}
```

2. Filtrando no Compass e no `mongosh`

Na barra **Filter** do Compass (ou dentro de `db.books.find(...)`):

```
// 1. Busca exata por atributo de topo:
{ "isbn": "101" }

// 2. Dot Notation em subdocumento:
{ "meta.version": "MongoDB 6.0" }

// 3. Operador de comparacao ($gt = maior que):
{ "price": { "$gt": 35 } }

// 4. Expressao Regular case-insensitive:
{ "name": { "$regex": "mongo", "$options": "i"
      } }
```

Passo 4: Atualizando (*Update*) e Removendo (*Delete*) Documentos

1. Atualizando com `updateOne` e `$set`

```
// Atualizar o preco do livro de ISBN "101":
db.books.updateOne(
  { "isbn": "101" },          // Filtro de
       busca
  { "$set": { "price": 45 } } // Operador
       atomico
)
// Retorno:
// { acknowledged: true,
//   matchedCount: 1, modifiedCount: 1 }
```

- **No Compass:** Passe o mouse sobre o documento, clique no lápis (**Edit Document**), altere `price` para `45`, clique em **Update** e consulte a lista de documentos!

2. Removendo com `deleteOne` e Lote (`bulkWrite`)

```
// Remover 1 documento especifico:
db.books.deleteOne({ "isbn": "201" })
// Retorno: { acknowledged: true,
       deletedCount: 1 }

// Operacao em Lote (Batch):
db.books.bulkWrite([
  { insertOne: { document: {
      isbn: "202", name: "MongoDB Guide",
            price: 40
  }}},
  { updateOne: {
      filter: { isbn: "101" },
      update: { "$set": { price: 49 } }
  }}
])
```

Programando com o Driver Oficial Python (PyMongo)

Conectando o Python ao MongoDB e Inserindo Documentos

1. Instalação e Teste de Conexão (`ping`)

```
# No terminal: pip install pymongo
from pymongo import MongoClient
from pprint import pprint

# Conectar ao servidor local na porta 27017:
uri = "mongodb://localhost:27017/"
client = MongoClient(uri)

# Confirmar conexao com o comando ping:
client.admin.command("ping")
print("Conectado ao MongoDB com sucesso!")

# Selecionar banco 'mongodb_books' e colecao
      'books'
books = client.mongodb_books.books
```

2. Inserindo com `insert_one` e `insert_many`

```
# Inserir 1 documento (dicionario Python):
book = {
    "isbn": "301",
    "name": "Python and MongoDB",
    "meta": {"version": "MongoDB 7.0"},
    "price": 60
}
res = books.insert_one(book)
print("ID gerado:", res.inserted_id)

# Inserir multiplos documentos em lote:
books.insert_many([
    {"isbn": "302", "name": "Python and
          MongoDB",
     "meta": {"version": "MongoDB 7.0"},
           "price": 60},
    {"isbn": "303", "name": "Advanced MongoDB
          Techniques",
     "price": 70}
])
```

Consultando Documentos no Python: `find`, Dot Notation, `$gt` e `$or`

1. Busca Simples, Dot Notation e `$regex`

```
# 1. Buscar pelo nome do livro:
for doc in books.find({"name": "Python and
       MongoDB"}):
    pprint(doc)

# 2. Buscar dentro do subdocumento 'meta'
       usando
# Dot Notation e Expressao Regular:
query_regex = {
    "meta.version": {
        "$regex": r".*?g.*?7\.0$",
        "$options": "i"
    }
}
for doc in books.find(query_regex):
    pprint(doc)
```

2. Comparação (`$gt`) e Operadores Lógicos

```
# 3. Livros com preco maior que 50 ($gt):
for doc in books.find({"price": {"$gt": 50}}):
    pprint(doc)

# 4. Condicao logica AND (multiplas chaves):
q_and = {"name": "Advanced MongoDB Techniques",
         "price": 70}
pprint(books.find_one(q_and))

# 5. Condicao logica OR ($or):
q_or = {
    "$or": [
        {"name": "Advanced MongoDB
              Techniques"},
        {"isbn": "301"}
    ]
}
for doc in books.find(q_or):
    pprint(doc)
```

Atualizando (`update_one`) e Removendo (`delete_one`) no Python

1. Atualizando com `update_one` e `$set`

```
# Atualizar o preco do livro "Advanced MongoDB
# Techniques" para 75:
update_result = books.update_one(
    {"name": "Advanced MongoDB Techniques"},
    {"$set": {"price": 75}}
)

print("Encontrados:",
      update_result.matched_count)
print("Modificados:",
      update_result.modified_count)

# Buscar o documento atualizado para confirmar:
updated_doc = books.find_one(
    {"name": "Advanced MongoDB Techniques"}
)
pprint(updated_doc)
```

2. Removendo com `delete_one`

```
# Remover o livro de ISBN '303':
isbn_to_delete = "303"
delete_result = books.delete_one(
    {"isbn": isbn_to_delete}
)
print("Removidos:",
      delete_result.deleted_count)

# Listando colecoes do banco via Python:
db = client.mongodb_books
print("Colecoes:", db.list_collection_names())
```

- **Sincronia com o Compass:** Clique em **Refresh** na aba **Documents** do Compass para ver as alterações!

Teoria de Schema Design: *Embedding*, *Referencing* e Padrões

Princípios de Schema Design no MongoDB

Três Estratégias Fundamentais

1. ***Embedded Data* (Dados Incorporados):** Aninhar objetos relacionados dentro do mesmo documento (ex.: um livro com o subdocumento `author`, ou uma cliente com a lista de `enderecos`).
2. ***References* (Referências):** Guardar o `ObjectId` de um documento em outra coleção (ex.: coleção `courses` apontando para `professor: ObjectId(...)`).
3. ***Denormalization* (Desnormalização):** Copiar campos específicos (como `authorName`) para evitar `$lookup` em leituras críticas.

A Métrica-Chave: *Read-Write Ratio*

- **Proporção Leitura vs. Escrita (*Read-Write Ratio*):**
  - **Cargas Intensivas em Leitura (*Read-Heavy*):** Catálogos de produtos e vitrines têm muito mais leituras que escritas → favorecem ***Embedding*** e **Desnormalização** (1 única busca traz tudo).
  - **Cargas Intensivas em Escrita (*Write-Heavy*):** Eventos frequentes ou entidades compartilhadas que mudam o tempo todo → favorecem ***Referencing***.
- **Regra de Ouro:** *"Dados lidos juntos pela aplicação devem ser armazenados juntos!"*

Visualizando as Duas Estratégias de Modelagem

1. Incorporação (*Embedding* / Desnormalizado)

Coleção: clientes (1 Único Documento)

```
{
  "_id": { "$oid": "671ec880...0010"},
  "nome": "Ana Costa",
  "cpf": "123.456.789-00",
  "enderecos": [ // Subdocumentos!
    { "apelido": "Casa", "rua": "Av. Chermont"},
    { "apelido": "IFCE", "rua": "CE-187, Taua"}
  ]
}
```

2. Referenciamento (*Referencing* / Normalizado)

Coleção: avaliacoes

```
{
  "_id": { "$oid": "671ec990...0999"},
  "produto_id": { "$oid": "671ec880...0001"},
  "autor": "Lucas", "nota": 5
}
```

Referência por `ObjectId`

Coleção: produtos

```
{
  "_id": { "$oid": "671ec880...0001"},
  "sku": "NOTE-ACER-15", "preco": 4599.90
}
```

O Anti-Padrão *Unbounded Arrays* e os Padrões de Projeto

Por que Evitar Arrays Ilimitados?

- Embutir listas sem fim (ex.: 50.000 avaliações no produto) estoura o limite de **16 MB** (`BSONObjectTooLarge`) e degrada o *WiredTiger*.
- **Cardinalidade: 1:1 / 1:Few (**<50**)** $\rightarrow$ *Embedding*; **1:Many** $\rightarrow$ *Referencing* / *Subset*; **1:Milhões** $\rightarrow$ Ref. no filho.

Padrões de Modelagem (*Design Patterns*)

Padrão	Aplicação Prática
Polymorphic	Categorias distintas na mesma coleção.
Subset	Últimas avaliações no produto.
Extended Ref	Campos do cliente/item no pedido.
Computed	Média e total pré-calculados.

Modelando o E-Commerce ShopGrids no Compass e no Python

Coleção `produtos`: *Polymorphic Pattern* e Validação

1. Documento 1: Eletrônicos (Compass)

```
{
  "_id": { "$oid": "671ec880a1b2c3d4e5f60001"
         },
  "sku": "NOTE-ACER-15",
  "nome": "Notebook Gamer Nitro V15",
  "categoria": "Eletronicos",
  "preco": { "$numberDecimal": "4599.90" },
  "estoque": 15,
  "especificacoes": {
    "cpu": "Intel Core i7-13620H",
    "ram_gb": 16,
    "gpu": "RTX 4050 6GB"
  },
  "garantia_meses": 12
}
```

2. Documento 2: Vestuário + Validação

```
{
  "_id": { "$oid": "671ec880a1b2c3d4e5f60002"
         },
  "sku": "CAM-TECH-ADS",
  "nome": "Camiseta Dev NoSQL IFCE",
  "categoria": "Vestuario",
  "preco": { "$numberDecimal": "69.90" },
  "estoque": 80,
  "especificacoes": {
    "tecido": "100% Algodao Egipcio",
    "cor": "Preta",
    "tamanhos": ["P", "M", "G", "GG"]
  }
}
```

- **Governança (`$jsonSchema` / Aba Validation):** Exige `sku`, `nome` e `preco > 0`, deixando `especificacoes` livre!

Coleções `avaliacoes` e `pedidos`: *Subset, Computed e Extended Reference*

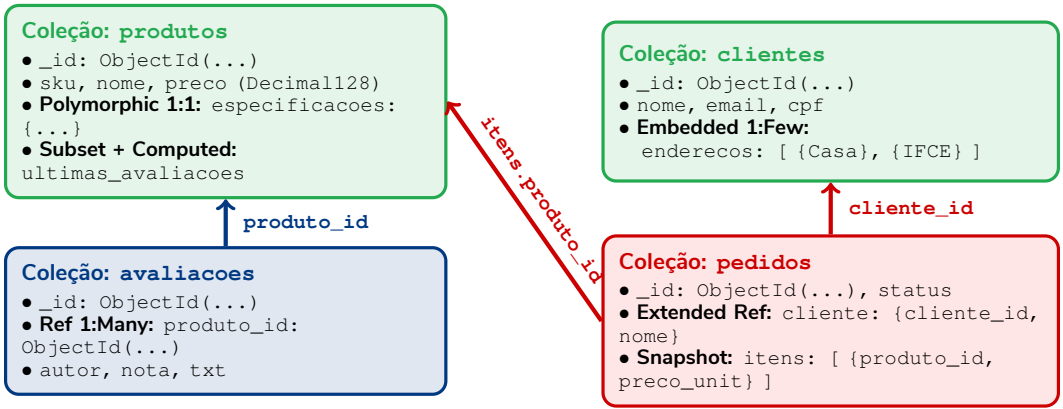
1. *Subset + Computed* (`avaliacoes`)

```
// Em produtos (Vitrine em 1 Query!):
{
  "_id": { "$oid": "671ec880a1b2c3d4e5f60001"
         },
  "sku": "NOTE-ACER-15",
  "resumo_avaliacoes": { "media": 5.0,
        "total": 2 },
  "ultimas_avaliacoes": [
    { "autor": "Lucas", "nota": 5, "txt":
        "Otimo!" }
  ]
}
// Em avaliacoes (Historico Referenciado):
{
  "_id": { "$oid": "671ec990a1b2c3d4e5f60999"
         },
  "produto_id": { "$oid":
       "671ec880a1b2c3d4e5f60001" },
  "autor": "Lucas", "nota": 5, "txt": "Otimo!"
}
```

2. *Extended Ref. + Snapshot* (`pedidos`)

```
{
  "_id": { "$oid": "671eca00a1b2c3d4e5f60777"
         },
  "codigo_pedido": "PED-2026-0001",
  "cliente": {
    "cliente_id": { "$oid": "671ec880...0010"
           },
    "nome": "Ana Costa", "cpf":
        "123.456.789-00"
  },
  "itens": [{
    "produto_id": { "$oid": "671ec880...0001"
           },
    "sku": "NOTE-ACER-15",
    "preco_unit": { "$numberDecimal":
        "4599.90" },
    "quantidade": 1
  }],
  "valor_total": { "$numberDecimal": "4599.90"
         },
  "status": "PAGO"
}
```

Arquitetura Completa das 4 Coleções do E-Commerce ShopGrids



Checklist de Decisão e Roteiro Passo a Passo (Compass + Python)

Pergunta de Projeto	*Embedding*	*Referencing*
1. Dados lidos juntos na mesma tela?	**Sim** (1 única query)	Não (telas separadas)
2. Qual a cardinalidade máxima?	**1:1** ou **1:Few** (< 50)	**1:Many** / **1:Milhões**
3. Array pode crescer sem limite?	Não (tamanho fixo)	**Sim** (risco de 16 MB)
4. Dado filho consultado isoladamente?	Raramente	**Sim, sempre**
5. Qual o *Read-Write Ratio*?	**Alta Leitura** (*Read-Heavy*)	**Alta Escrita** (*Write-Heavy*)
6. Congelar valor fiscal histórico?	**Sim (*Snapshot*)**	Não (aponta para atual)

Sequência da Prática: Etapa 1: CRUD do Zero em `mongodb_books` (Compass + `mongosh`) → **Etapa 2:** CRUD com Python (`PyMongo`) → **Etapa 3:** Modelagem manual do 1.º registro de cada coleção em `ecommerce_shopgrids` no Compass → **Etapa 4:** Povoamento em lote via Compass/Python.

Dúvidas & Prática em Laboratório

IFCE Campus Tauá – ADS16 (Modelo Orientado a Documentos)

Do CRUD Básico ao Python (`PyMongo`) e Modelagem BSON no MongoDB